

Figure 1: Component stack for Big Data processing

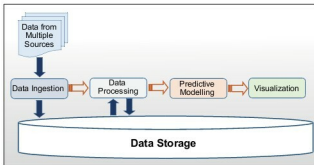


Figure 2: Big Data processing pipeline

This article aims to provide an overview of the frameworks and components available in open source, across different layers of a Big Data processing stack.

Component architecture stack for Big Data processing

As more and more Big Data, characterised by the three Vs - volume, velocity and variety - began to be generated and acquired, different systems started to evolve to tap the vast and diverse potential of this data. Although some of the systems converged in terms of the features they offered, they were all driven by different underlying design philosophies and, therefore, offered different alternatives. However, one of the guiding principles to develop enterprise data strategies would be to have a generic data storage layer as a data lake, which would allow different computing frameworks to work on the data stored for different processing use cases, and have the data shared across frameworks. Figure 1 illustrates a representational architectural stack for Big Data processing.

The stack can also be visualised as a pipeline consisting of multiple stages through which the data is driven, as can be seen in Figure 2. The unstructured and often schemaless raw data that is sourced from multiple sources, such as transactions, weblogs, open social sites, other linked data sources and databases, devices and instruments, could be in varying formats – such as textual data, images, video, audio,

etc. This data is then cleaned and often checked for any errors when ingested into the data storage layer. It is then processed, a task that might occur iteratively and interactively using the frameworks described in later sections. The processing could itself have multiple sub-stages and could revert data back into the storage layer at every iteration. It could be further explored and modelled using statistical algorithms to derive and validate hypotheses. These algorithms are trained on the data to learn the models, which can then be used for predictive modelling. The models could then be trained periodically as newer datasets flow into the system. The datasets are further used for exploratory analytics to discover unseen intelligence and insights. During the processing and exploratory processes, the processed datasets are visualised using visualisation tools to aid data understanding and for communicating to stakeholders.

This data in the storage layer could be reused by different stakeholders within an organisation. Big Data is typically undefined and most frameworks, as we will see later, have adapted to this aspect of it. In fact, this very feature is instrumental in the success of a framework.

Let us discuss some of the frameworks and libraries across these different layers.

The storage and data layer

Let's start with the storage and data layer, which is the most critical and the foundation of a Big Data stack. Big Data is typically characterised by its volume, requiring huge and conceptually unlimited storage capacities. Advances in technology, contributing to cheaper storage and compute resources, have resulted in the emergence of cluster storage and compute platforms. The platforms have unlocked the storage limitations and virtually enabled unlimited amounts of data storage. These platforms are not limited by the traditional paradigms of data modelling and schema designs. They are generally schema-free and allow the storage of all forms of data (structured, semi-structured and unstructured). This enables the creation of systems that are more dynamic and which enable analysts to explore the data without being limited by preconceived models. In this section, we will look at some of the popular cluster storage frameworks for Big Data.

HDFS (<https://hadoop.apache.org/>): This is a scalable, fault-tolerant distributed file system in the Hadoop ecosystem. HDFS is scaled by adding commodity servers into the clusters. The largest cluster size is known to be about 4500 nodes in a cluster with up to 128 petabytes of data. HDFS supports parallel reading and writing of data. The bandwidth in an HDFS system scales linearly with the number of nodes. There is built-in redundancy with multiple copies of data stored in the system. The files are broken into blocks and stored as files across the cluster. They are replicated for reliability.